# Parallel Programming using RDD

Sure! Let's talk about Resilient Distributed Datasets (RDDs) in simple terms.

**Understanding RDDs: A Beginner's Guide**

**Resilient Distributed Datasets**, or RDDs, are like a special type of data collection used in Apache Spark, which is a powerful tool for processing large amounts of data. Imagine you have a big box of LEGO bricks (the data) that you want to build something with. Instead of trying to build it all by yourself, you can share the bricks with your friends (the nodes in a cluster) so that everyone can work on different parts of the project at the same time. This is what RDDs do—they allow data to be split up and processed in parallel across multiple computers, making the whole process faster and more efficient.

**Why RDDs are Resilient**

Now, what makes RDDs "resilient"? Think of it like this: if one of your friends accidentally drops their LEGO section and it breaks, you still have the original pieces in the box. RDDs are designed to be fault-tolerant, meaning that if something goes wrong, the data can be recovered easily because it is immutable—once you create it, you can't change it. This ensures that your data is safe and can be accessed again, just like having a backup of your LEGO bricks.

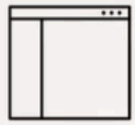
## What are RDDs

A resilient distributed dataset is:
- Spark's primary data abstraction
- A fault-tolerant collection of elements
- Partitioned across the nodes of the cluster
- Capable of accepting parallel operations
- Immutable

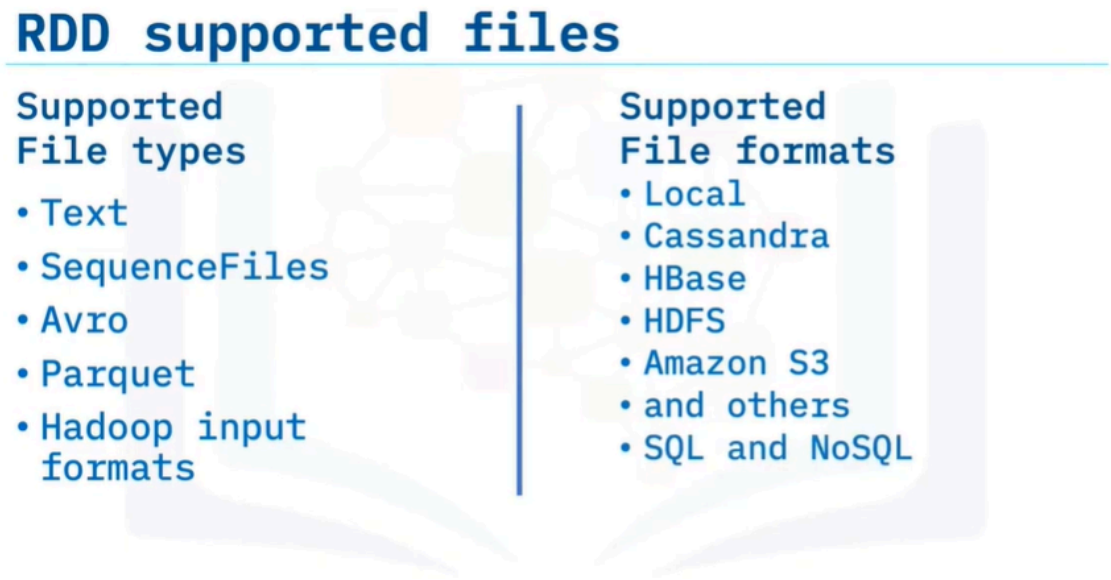
- A Resilient Distributed Dataset, also known as an RDD, is Spark's primary data abstraction. A Resilient Distributed Dataset is a collection of fault-tolerant elements partitioned across the cluster's nodes, capable of receiving parallel operations. Additionally, Resilient Distributed Datasets are immutable, meaning that these datasets cannot be changed once created.

## Spark applications

Consist of a driver program that runs
The user's main functions
And multiple *parallel operations*
on a cluster.



- Every Spark application consists of a driver program that runs the user's main functions and runs multiple parallel operations on a cluster.

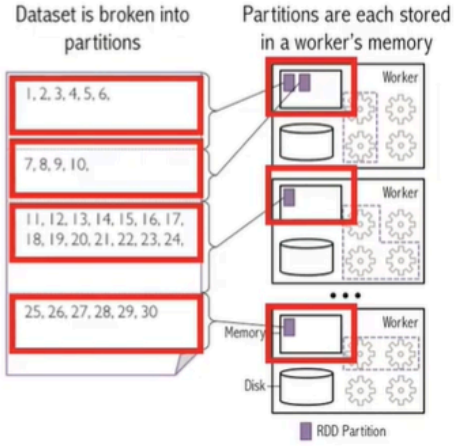



- RDDs support text, sequence files, Avro, Parquet, and Hadoop input format file types. RDDs also support local, Cassandra, HBase, HDFS, Amazon S3, and other file formats in addition to an abundance of relational and NoSQL databases.

## Creating an RDD in Spark

Use an external or local file from Hadoop-supported file system such as:

- HDFS
- Cassandra
- Hbase
- Amazon S3




- First, you can create an RDD using an external or local file from a Hadoop-supported file system such as HDFS, Cassandra, HBase, or Amazon S3.

## Create an RDD from a collection

Simple examples of creating a RDD from a list in Scala and Python

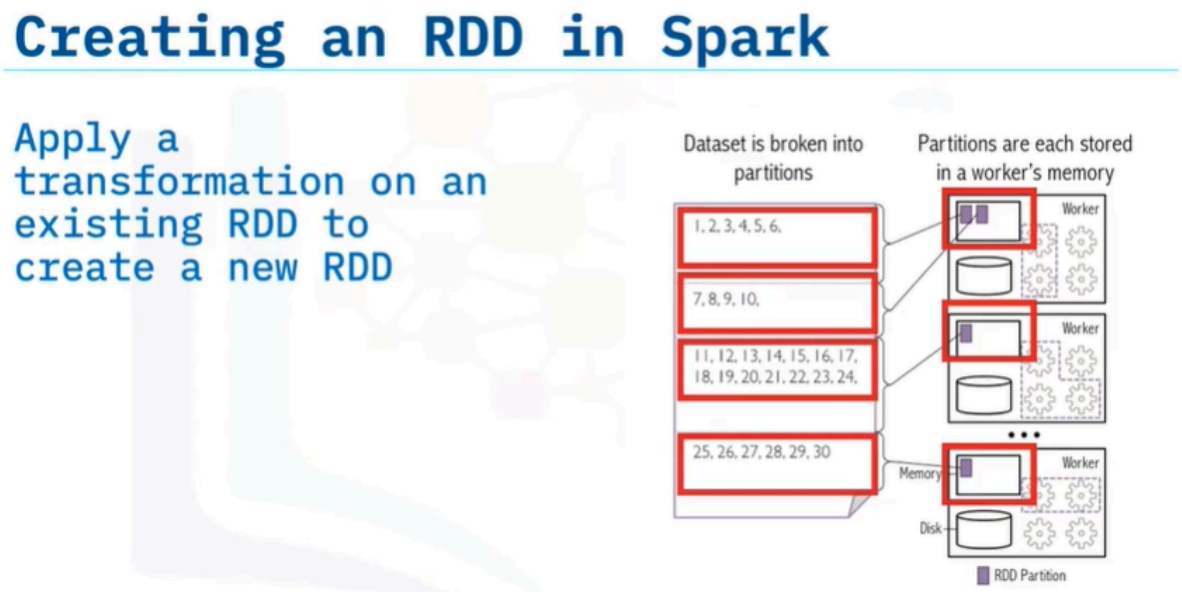
```
// Scala example
val data = Array(1, 2, 3, 4, 5)
val distData = sc.parallelize(data)
```

```
// Python example
data = [1, 2, 3, 4, 5]
distData = sc.parallelize(data)
```

- A second method is to apply the parallelize function to an existing collection in the driver program. Note that this driver program can be in any of the supported high-level APIs, including Python, Java, and Scala. This simple

code snippet, shown on screen, using both Scala and Python, creates an RDD from an existing collection. In this case, we use a list. You'll see SC applied to the snippet. One important parameter for parallel collections is the number of partitions specified to cut the dataset. Spark runs one task for each partition of the cluster. Typically, you want two to four partitions for each CPU in your cluster. Spark tries to automatically set the number of partitions based on your cluster.

- However, you can also set partitions manually by passing the number as a second parameter to the parallelize function.




- A third method of creating an RDD in Spark is to apply a transformation on an existing RDD to create a new RDD.

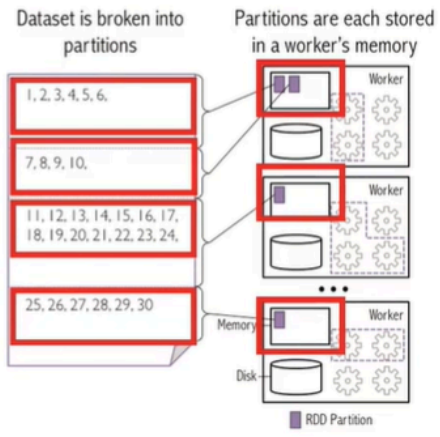
## What is Parallel Programming

- Parallel programming:
- Is the simultaneous use of multiple compute resources to solve a computational problem
- Breaks problems into discrete parts that can be solved concurrently
- Runs simultaneous instructions on multiple processors
- Employs an overall control/coordination mechanism

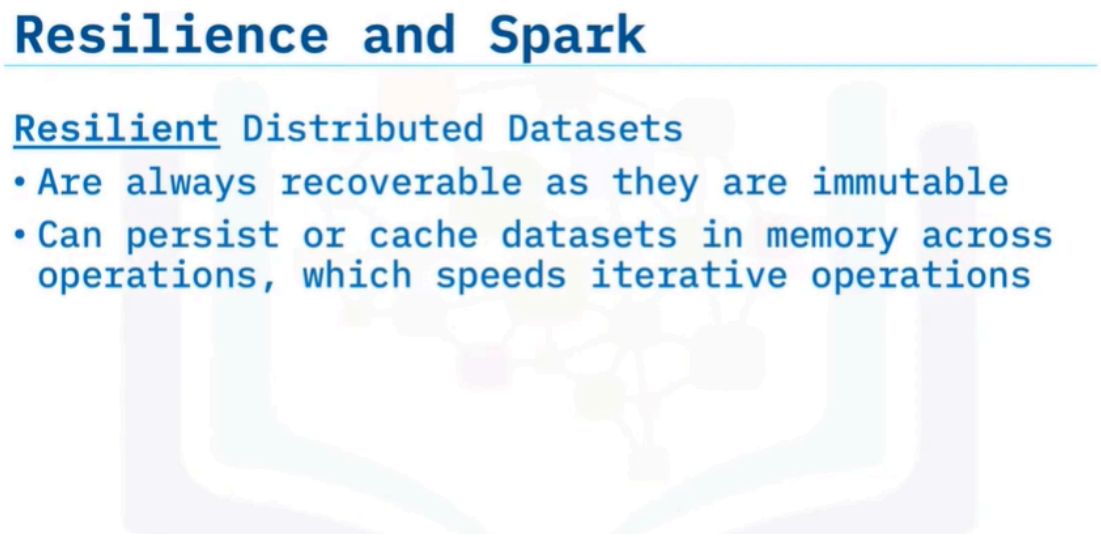
- Next, let's define parallel programming. Parallel programming, like distributed programming, is the simultaneous use of multiple compute resources to solve a computational task. Parallel programming parses tasks into discrete parts that are solved concurrently using multiple processors. The processors access a shared pool of memory, which has in place control and coordination mechanisms.

## RDDs & Parallel Programming

- You can create an RDD by parallelizing an array of objects, or by splitting a dataset into partitions
- Spark runs one task for each partition of the cluster



- Here is an example of how RDDs enable parallel programming. This example is similar to creating an RDD in Spark. You create an RDD by parallelizing an array of objects or by splitting a dataset into partitions. Nodes receive the distributed partitions. Spark runs one task for each partition of the cluster. Based on how RDDs are created, RDDs can inherently be operated on in parallel.




- So, how do RDDs provide resilience in Spark? RDDs provide resilience in Spark through immutability and caching. First, RDDs are always recoverable as the data is immutable. Another essential Spark capability is the persisting, or caching, of a dataset in memory across operations. The cache is fault-tolerant and always recoverable as RDDs are immutable and the Hadoop data source is also fault-tolerant. When you persist an RDD, each node stores the partitions that the node computed in memory and reuses the same partition in other actions on that dataset or the subsequent datasets derived from the first RDD.
- Persistence allows future actions to be much faster, often by more than 10 times. Persisting, or caching, is used as a key tool for iterative algorithms and fast, interactive use.

# Summary

In this video, you learned that:

- You can create an RDD using an external or local file from Hadoop-supported file, from a collection or from another RDD
- RDDs are immutable and always recoverable
- RDDs can persist or cache datasets in memory across operations, which speeds iterative operations